```
# ==============================================
# 1. COMPLETE IMPORTS & SETUP
# ==============================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV, cross_val_score
from sklearn.preprocessing import OneHotEncoder, StandardScaler, LabelEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.feature_selection import (
    SelectKBest,
    SelectFromModel,
    RFE,
    mutual_info_classif,
    chi2,
    VarianceThreshold
)
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA

# Models
from sklearn.linear_model import LogisticRegression, SGDClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

# Imbalance
from imblearn.over_sampling import SMOTEN
from imblearn.pipeline import Pipeline as ImbPipeline

# Metrics
from sklearn.metrics import (
    roc_auc_score, roc_curve,
    confusion_matrix, classification_report,
    precision_recall_curve, average_precision_score
)

# Interpretation
import shap

# Warnings
import warnings
warnings.filterwarnings('ignore')

# Set random seed
np.random.seed(42)

print("✅ All libraries imported successfully!")
```

```
✅ All libraries imported successfully!
```

```
# ==============================================
# 1. LOAD AND PREPARE DATA
# ==============================================

# Load your data
data = pd.read_stata(r"C:\Users\hp\Documents\Materials\Machine Learning\data\imputeddata3.dta")

# Encode target
data['wasting'] = data['wasting'].map({'No': 0, 'Yes': 1})
target = 'wasting'

# Split features and target
y = data[target]
X = data.drop(columns=[target])

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)

print("\n📊 DATA LOADED:")
print(f"   Train set: {X_train.shape[0]:,} samples")
print(f"   Test set: {X_test.shape[0]:,} samples")
print(f"   Original features: {X_train.shape[1]}")
```

```
📊 DATA LOADED:
   Train set: 60,979 samples
   Test set: 15,245 samples
   Original features: 26
```

```
# ==============================================
# 2. IDENTIFY COLUMN TYPES
# ==============================================

categorical_cols = X_train.select_dtypes(include=['object', 'category']).columns.tolist()
numeric_cols = X_train.select_dtypes(include=['int64', 'float64']).columns.tolist()

print(f"\n📋 FEATURE TYPES:")
print(f"   Categorical: {len(categorical_cols)} columns")
print(f"   Numeric: {len(numeric_cols)} columns")
```

```
📋 FEATURE TYPES:
```

```
   Categorical: 26 columns
   Numeric: 0 columns
```

```
# ==============================================
# 3. CREATE PREPROCESSOR WITH PROPER FEATURE NAMES
# ==============================================

preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), categorical_cols),
        ('num', StandardScaler(), numeric_cols)
    ],
    remainder='drop',
    verbose_feature_names_out=False
)

# Fit and transform
X_train_enc = preprocessor.fit_transform(X_train)
X_test_enc = preprocessor.transform(X_test)

# Get feature names
def get_feature_names(column_transformer, input_features):
    feature_names = []
    for name, transformer, columns in column_transformer.transformers_:
        if name == 'remainder' and transformer == 'drop':
            continue
        if hasattr(transformer, 'get_feature_names_out'):
            if name == 'cat':
                cat_names = transformer.get_feature_names_out(columns)
                feature_names.extend(cat_names)
            else:
                feature_names.extend(columns)
        else:
            if name == 'cat':
                for col in columns[:10]:  # Limit for fallback
                    feature_names.append(f"{col}_encoded")
            else:
                feature_names.extend(columns)
    return feature_names

feature_names = get_feature_names(preprocessor, X_train.columns.tolist())

# Create DataFrames
X_train_enc = pd.DataFrame(X_train_enc, columns=feature_names)
X_test_enc = pd.DataFrame(X_test_enc, columns=feature_names)

print(f"\n✅ ENCODING COMPLETE:")
print(f"   Encoded features: {X_train_enc.shape[1]}")
```

```
✅ ENCODING COMPLETE:
   Encoded features: 78
```

```
# ==============================================
# 4. APPLY SMOTEN FOR IMBALANCED DATA
# ==============================================

smote = SMOTEN(random_state=42, k_neighbors=3)
X_train_bal, y_train_bal = smote.fit_resample(X_train_enc, y_train)
X_train_bal = pd.DataFrame(X_train_bal, columns=feature_names)

print(f"\n⚖️ SMOTEN APPLIED:")
print(f"   Before: {y_train.value_counts().to_dict()}")
print(f"   After: {pd.Series(y_train_bal).value_counts().to_dict()}")
```

```
⚖️ SMOTEN APPLIED:
   Before: {0: 57273, 1: 3706}
   After: {0: 57273, 1: 57273}
```

```
# ==============================================
# 5. APPLY MULTIPLE FEATURE SELECTION METHODS
# ==============================================

print("\n" + "="*80)
print("🔍 APPLYING FEATURE SELECTION METHODS")
print("="*80)

feature_selection_results = {}

# 5.1 Variance Threshold
print("\n📌 Variance Threshold...")
selector_var = VarianceThreshold(threshold=0.01)
selector_var.fit(X_train_bal)
var_features = np.array(feature_names)[selector_var.get_support()]
feature_selection_results['variance_threshold'] = {
    'features': var_features,
    'n_features': len(var_features)
}
print(f"   ✅ Selected {len(var_features)} features")

# 5.2 Mutual Information
print("\n📌 Mutual Information...")
n_features_mi = min(50, X_train_bal.shape[1])
selector_mi = SelectKBest(score_func=mutual_info_classif, k=n_features_mi)
selector_mi.fit(X_train_bal, y_train_bal)
mi_features = np.array(feature_names)[selector_mi.get_support()]
feature_selection_results['mutual_info'] = {
    'features': mi_features,
    'n_features': len(mi_features)
}
print(f"   ✅ Selected {len(mi_features)} features")
```

```
# 5.3 Chi-Square
print("\n📌 Chi-Square...")
# Ensure non-negative for chi2
X_train_nonneg = X_train_bal.copy()
X_train_nonneg[X_train_nonneg < 0] = 0
n_features_chi2 = min(50, X_train_bal.shape[1])
selector_chi2 = SelectKBest(score_func=chi2, k=n_features_chi2)
selector_chi2.fit(X_train_nonneg, y_train_bal)
chi2_features = np.array(feature_names)[selector_chi2.get_support()]
feature_selection_results['chi2'] = {
    'features': chi2_features,
    'n_features': len(chi2_features)
}
print(f"   ✅ Selected {len(chi2_features)} features")

# 5.4 SelectFromModel (Random Forest)
print("\n📌 SelectFromModel (Random Forest)...")
rf_selector = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf_selector.fit(X_train_bal, y_train_bal)
selector_rf = SelectFromModel(rf_selector, threshold='median', prefit=True)
rf_features = np.array(feature_names)[selector_rf.get_support()]
feature_selection_results['select_from_model'] = {
    'features': rf_features,
    'n_features': len(rf_features)
}
print(f"   ✅ Selected {len(rf_features)} features")

# 5.5 Recursive Feature Elimination (RFE)
print("\n📌 Recursive Feature Elimination...")
n_features_rfe = min(40, X_train_bal.shape[1])
estimator = LogisticRegression(max_iter=1000, random_state=42)
selector_rfe = RFE(estimator, n_features_to_select=n_features_rfe, step=5)
selector_rfe.fit(X_train_bal, y_train_bal)
rfe_features = np.array(feature_names)[selector_rfe.get_support()]
feature_selection_results['rfe'] = {
    'features': rfe_features,
    'n_features': len(rfe_features)
}
print(f"   ✅ Selected {len(rfe_features)} features")

print("\n✅ ALL FEATURE SELECTION METHODS COMPLETED")
```

```
================================================================================
🔍 APPLYING FEATURE SELECTION METHODS
================================================================================

📌 Variance Threshold...
   ✅ Selected 75 features

📌 Mutual Information...
   ✅ Selected 50 features

📌 Chi-Square...
   ✅ Selected 50 features

📌 SelectFromModel (Random Forest)...
   ✅ Selected 39 features

📌 Recursive Feature Elimination...
   ✅ Selected 40 features

✅ ALL FEATURE SELECTION METHODS COMPLETED
```

```
# ==============================================
# 6. CREATE CONSENSUS FEATURE SET (SELECTED BY ≥2 METHODS)
# ==============================================

print("\n" + "="*80)
print("🎯 CREATING CONSENSUS FEATURE SET (≥2 METHODS)")
print("="*80)

# Count votes for each feature
feature_votes = {}
selection_methods = ['mutual_info', 'rfe', 'chi2', 'select_from_model', 'variance_threshold']

for method in selection_methods:
    if method in feature_selection_results:
        features = feature_selection_results[method]['features']
        for feature in features:
            feature_votes[feature] = feature_votes.get(feature, 0) + 1

print(f"\n📊 VOTE DISTRIBUTION:")
vote_distribution = {}
for votes in feature_votes.values():
    vote_distribution[votes] = vote_distribution.get(votes, 0) + 1

for votes in sorted(vote_distribution.keys()):
    percentage = (vote_distribution[votes] / len(feature_votes)) * 100
    bar = '█' * int(percentage / 5)
    print(f"   {votes} method(s): {vote_distribution[votes]:5d} features ({percentage:5.1f}%) {bar}")

# Get features selected by at least 2 methods
consensus_features = [
    feature for feature, votes in feature_votes.items()
    if votes >= 2
]

print(f"\n✅ CONSENSUS FEATURE SET:")
print(f"   Total features in consensus set: {len(consensus_features):,}")
print(f"   Original encoded features: {X_train_bal.shape[1]:,}")
print(f"   Reduction: {(1 - len(consensus_features)/X_train_bal.shape[1])*100:.1f}%")

# Show top consensus features
print(f"\n🏆 TOP CONSENSUS FEATURES:")
top_consensus = sorted(feature_votes.items(), key=lambda x: x[1], reverse=True)[:15]
```

```
for i, (feature, votes) in enumerate(top_consensus, 1):
    short_feature = feature[:60] + '...' if len(feature) > 60 else feature
    print(f"   {i:2d}. {short_feature:<60} (voted by {votes} methods)")
```

```
================================================================================
🎯 CREATING CONSENSUS FEATURE SET (≥2 METHODS)
================================================================================

📊 VOTE DISTRIBUTION:
   1 method(s):     6 features (  7.8%) ▌
   2 method(s):    23 features ( 29.9%) ▌▌▌▌
   3 method(s):    11 features ( 14.3%) ▌▌
   4 method(s):    16 features ( 20.8%) ▌▌▌
   5 method(s):    21 features ( 27.3%) ▌▌▌▌

✅ CONSENSUS FEATURE SET:
   Total features in consensus set: 71
   Original encoded features: 78
   Reduction: 9.0%

🏆 TOP CONSENSUS FEATURES:
    1. matage_15-19                                            (voted by 5 methods)
    2. matage_20-24                                            (voted by 5 methods)
    3. matage_25-29                                            (voted by 5 methods)
    4. matage_30-34                                            (voted by 5 methods)
    5. matage_35-39                                            (voted by 5 methods)
    6. matage_40-44                                            (voted by 5 methods)
    7. placeresid_rural                                        (voted by 5 methods)
    8. famsize_1-5                                             (voted by 5 methods)
    9. famsize_6 and more                                      (voted by 5 methods)
   10. num5child_3 and more                                    (voted by 5 methods)
   11. childgender_female                                      (voted by 5 methods)
   12. childage_12-23 months                                   (voted by 5 methods)
   13. childage_24-35 months                                   (voted by 5 methods)
   14. childage_36-47 months                                   (voted by 5 methods)
   15. childage_48-59 months                                   (voted by 5 methods)
```

```
# ==============================================
# 7. CREATE CONSENSUS DATASET
# ==============================================

# Get indices of consensus features
consensus_indices = []
valid_consensus_features = []

for feature in consensus_features:
    try:
        idx = list(feature_names).index(feature)
        consensus_indices.append(idx)
        valid_consensus_features.append(feature)
    except ValueError:
        continue

# Create consensus datasets
X_train_consensus = X_train_bal.values[:, consensus_indices]
X_test_consensus = X_test_enc.values[:, consensus_indices]

# Convert to DataFrame
X_train_consensus_df = pd.DataFrame(X_train_consensus, columns=valid_consensus_features)
X_test_consensus_df = pd.DataFrame(X_test_consensus, columns=valid_consensus_features)

print(f"\n✅ CONSENSUS DATASET CREATED:")
print(f"   Train shape: {X_train_consensus_df.shape}")
print(f"   Test shape: {X_test_consensus_df.shape}")
```

```
✅ CONSENSUS DATASET CREATED:
   Train shape: (114546, 71)
   Test shape: (15245, 71)
```

```
# ==============================================
# 8. TRAIN ALL MODELS ON CONSENSUS FEATURES ONLY
# ==============================================

print("\n" + "="*80)
print("🚀 TRAINING ALL MODELS ON CONSENSUS FEATURES")
print("="*80)

# Calculate scale_pos_weight
scale_pos = len(y_train_bal[y_train_bal == 0]) / len(y_train_bal[y_train_bal == 1])

# Define models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, solver='liblinear', random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=500, random_state=42, n_jobs=-1),
    "Decision Tree": DecisionTreeClassifier(max_depth=10, random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=15, n_jobs=-1),
    "Naive Bayes": GaussianNB(),
    "SVM": SVC(probability=True, class_weight='balanced', random_state=42),
    "XGBoost": XGBClassifier(scale_pos_weight=scale_pos, eval_metric='logloss',
                             random_state=42, verbosity=0, n_jobs=-1),
    "LightGBM": LGBMClassifier(class_weight='balanced', random_state=42, verbose=-1, n_jobs=-1),
    "CatBoost": CatBoostClassifier(class_weights=[1, scale_pos], random_state=42, verbose=0),
    "SGD": SGDClassifier(loss='log_loss', class_weight='balanced', random_state=42, n_jobs=-1)
}

# Train and evaluate
consensus_models = {}
consensus_results = []

for name, model in models.items():
    print(f"\n📊 Training: {name:<20}...", end="")

    try:
        model.fit(X_train_consensus_df, y_train_bal)
```

```
            consensus_models[name] = model

            # Predictions
            y_pred_proba = model.predict_proba(X_test_consensus_df)[:, 1]

            # Metrics
            auc = roc_auc_score(y_test, y_pred_proba)
            pr_auc = average_precision_score(y_test, y_pred_proba)

            # Optimal threshold
            fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
            youden_j = tpr - fpr
            optimal_idx = np.argmax(youden_j)
            optimal_threshold = thresholds[optimal_idx]

            # Confusion Matrix
            y_pred_optimal = (y_pred_proba >= optimal_threshold).astype(int)
            cm = confusion_matrix(y_test, y_pred_optimal)

            consensus_results.append({
                'model': name,
                'auc': auc,
                'pr_auc': pr_auc,
                'optimal_threshold': optimal_threshold,
                'fpr': fpr,
                'tpr': tpr,
                'confusion_matrix': cm,
                'n_features': len(valid_consensus_features)
            })

            print(f" ✓ AUC: {auc:.4f}")

        except Exception as e:
            print(f" ✗ Failed: {str(e)[:50]}")

    # Convert to DataFrame
    consensus_results_df = pd.DataFrame(consensus_results)
    consensus_results_df = consensus_results_df.sort_values('auc', ascending=False)

    print("\n" + "="*80)
    print("📊 MODEL PERFORMANCE ON CONSENSUS FEATURES")
    print("="*80)
    print(f"\n{'Rank':<6} {'Model':<20} {'AUC':<10} {'PR-AUC':<10} {'Threshold':<10}")
    print("-" * 60)
    for i, row in consensus_results_df.iterrows():
        rank = i + 1
        print(f"{rank:<6} {row['model']:<20} {row['auc']:.4f}     {row['pr_auc']:.4f}     {row['optimal_threshold']:.4f}")

    # Best model
    best_model_name = consensus_results_df.iloc[0]['model']
    best_model = consensus_models[best_model_name]
    best_auc = consensus_results_df.iloc[0]['auc']

    print("\n" + "✨"*50)
    print(f"✨ BEST MODEL: {best_model_name}")
    print(f"✨ AUC: {best_auc:.4f}")
    print(f"✨ Features: {len(valid_consensus_features)}")
    print("✨"*50)
```

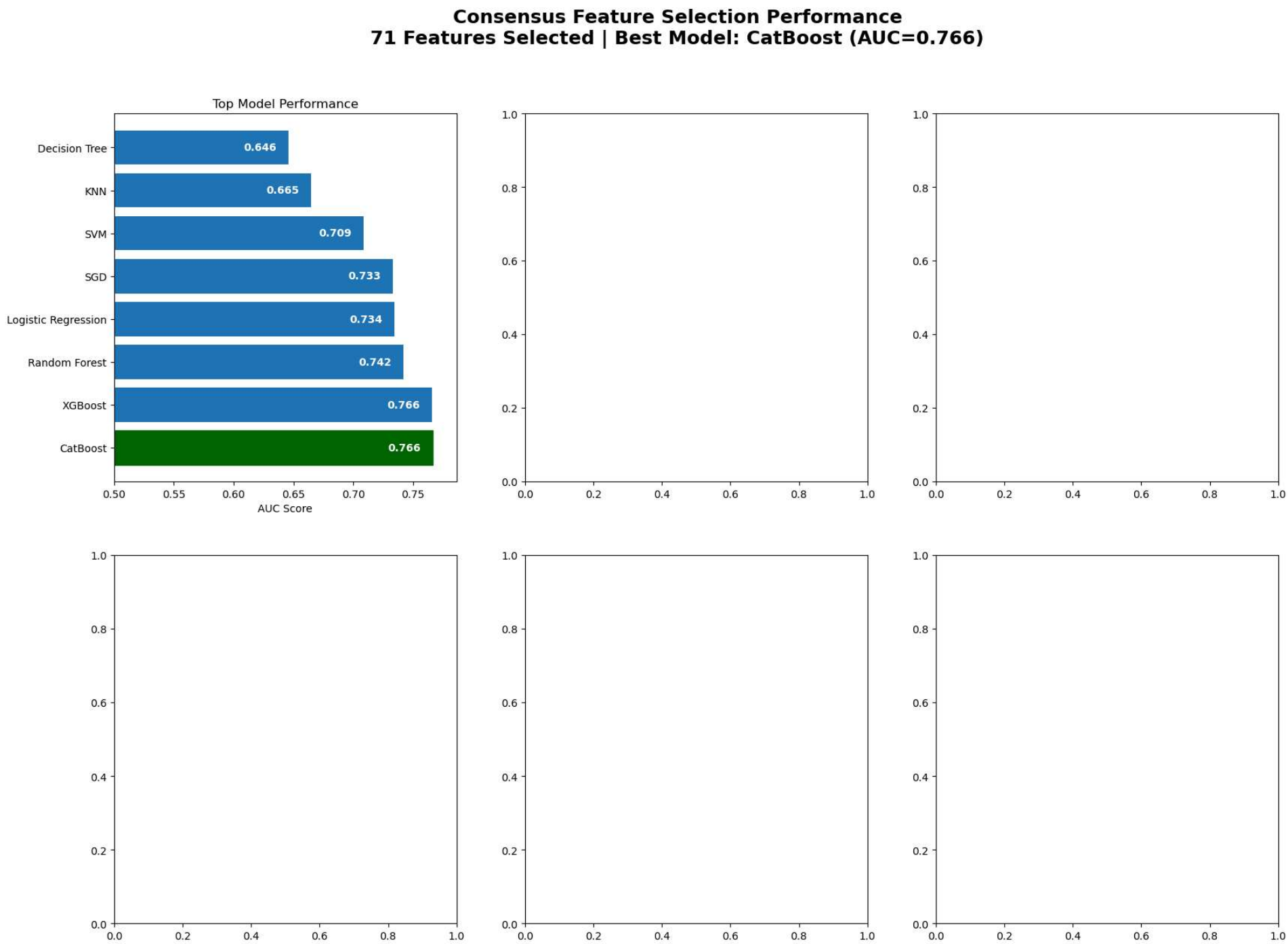
```
================================================================================
🚀 TRAINING ALL MODELS ON CONSENSUS FEATURES
================================================================================

📊 Training: Logistic Regression ... ✓ AUC: 0.7343

📊 Training: Random Forest       ... ✓ AUC: 0.7420

📊 Training: Decision Tree       ... ✓ AUC: 0.6462

📊 Training: KNN                 ... ✓ AUC: 0.6646

📊 Training: Naive Bayes         ... ✓ AUC: 0.6370

📊 Training: SVM                 ... ✓ AUC: 0.7090

📊 Training: XGBoost             ... ✓ AUC: 0.7659

📊 Training: LightGBM            ... ✗ Failed: Do not support special JSON characters in feature

📊 Training: CatBoost            ... ✓ AUC: 0.7665

📊 Training: SGD                 ... ✓ AUC: 0.7331

================================================================================
📊 MODEL PERFORMANCE ON CONSENSUS FEATURES
================================================================================

Rank   Model                AUC        PR-AUC     Threshold
------------------------------------------------------------
8      CatBoost             0.7665     0.1933     0.0555
7      XGBoost              0.7659     0.1921     0.0570
2      Random Forest        0.7420     0.1691     0.0940
1      Logistic Regression  0.7343     0.1427     0.0927
9      SGD                  0.7331     0.1402     0.1300
6      SVM                  0.7090     0.1371     0.0562
4      KNN                  0.6646     0.1054     0.1333
3      Decision Tree        0.6462     0.1064     0.0708
5      Naive Bayes          0.6370     0.1215     0.8259

✨✨✨
```

```
    new_columns = []

    for col in df.columns:
        # Remove JSON-problematic characters
        clean_col = re.sub(r'[^A-Za-z0-9_]', '_', col)
        new_columns.append(clean_col)

    df.columns = new_columns
    return df

# Clean feature names for LightGBM
X_train_lgb = clean_feature_names(X_train_consensus_df)
X_test_lgb = clean_feature_names(X_test_consensus_df)

from lightgbm import LGBMClassifier
from sklearn.metrics import roc_auc_score, average_precision_score

# Calculate imbalance ratio
scale_pos_weight = (y_train_bal == 0).sum() / (y_train_bal == 1).sum()

lgb_model = LGBMClassifier(
    n_estimators=800,
    learning_rate=0.03,
    max_depth=-1,
    num_leaves=31,
    min_child_samples=20,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=scale_pos_weight,
    random_state=42,
    n_jobs=-1
)

lgb_model.fit(X_train_lgb, y_train_bal)

# Predictions
y_pred_proba = lgb_model.predict_proba(X_test_lgb)[:, 1]

auc = roc_auc_score(y_test, y_pred_proba)
pr_auc = average_precision_score(y_test, y_pred_proba)

print("\n🎯 LightGBM Results")
print(f"AUC: {auc:.4f}")
print(f"PR-AUC: {pr_auc:.4f}")
```

```
🎯 LightGBM Results
AUC: 0.7826
PR-AUC: 0.1942
```

```
# ============================================
# 9. VISUALIZE RESULTS (UPDATED VERSION)
# ============================================

import numpy as np
import matplotlib.pyplot as plt

# Ensure results are sorted
consensus_results_df = consensus_results_df.sort_values(
    'auc', ascending=False
).reset_index(drop=True)

best_auc = consensus_results_df.iloc[0]['auc']
best_model_name = consensus_results_df.iloc[0]['model']

fig, axes = plt.subplots(2, 3, figsize=(20, 14))

fig.suptitle(
    f'Consensus Feature Selection Performance\n'
    f'{len(valid_consensus_features)} Features Selected | Best Model: {best_model_name} (AUC={best_auc:.3f})',
    fontsize=18,
    fontweight='bold'
)

# ============================================
# 9.1 Model AUC Comparison
# ============================================
ax1 = axes[0, 0]
top_models = consensus_results_df.head(8)

bars = ax1.barh(
    range(len(top_models)),
    top_models['auc'].values
)

ax1.set_yticks(range(len(top_models)))
ax1.set_yticklabels(top_models['model'].values)
ax1.set_xlabel('AUC Score')
ax1.set_title('Top Model Performance')

# Dynamic axis scaling
ax1.set_xlim([0.5, top_models['auc'].max() + 0.02])

# Highlight best model
bars[0].set_color('darkgreen')

for bar, auc in zip(bars, top_models['auc'].values):
    ax1.text(
        bar.get_width() - 0.01,
        bar.get_y() + bar.get_height()/2,
        f'{auc:.3f}',
        ha='right',
        va='center',
        color='white',
```

```
        fontweight='bold'
    )
```




```
# ==============================================
# 9.2 ROC Curves (Top 4 models only if stored)
# ==============================================
ax2 = axes[0, 1]

for idx, row in consensus_results_df.head(4).iterrows():
    if row.get('fpr') is not None and row.get('tpr') is not None:
        ax2.plot(
            row['fpr'],
            row['tpr'],
            linewidth=2,
            label=f"{row['model']} (AUC={row['auc']:.3f})"
        )

ax2.plot([0, 1], [0, 1], linestyle='--', alpha=0.4)
ax2.set_xlabel('False Positive Rate')
ax2.set_ylabel('True Positive Rate')
ax2.set_title('ROC Curves - Top Models')
ax2.legend(loc='lower right')
ax2.grid(alpha=0.3)
```

```
# ==============================================
# 9.3 Vote Distribution
# ==============================================
ax3 = axes[0, 2]

votes_counts = sorted(vote_distribution.keys())
freq_counts = [vote_distribution[v] for v in votes_counts]

bars = ax3.bar(votes_counts, freq_counts)

ax3.set_xlabel('Number of Methods Selecting Feature')
ax3.set_ylabel('Number of Features')
ax3.set_title('Feature Consensus Distribution')

ax3.axvspan(1.5, max(votes_counts)+0.5, alpha=0.15, color='green', label='Consensus (≥2)')
ax3.legend()

for bar, freq in zip(bars, freq_counts):
    ax3.text(
        bar.get_x() + bar.get_width()/2,
        bar.get_height() + 2,
        str(freq),
        ha='center',
        fontweight='bold'
    )
```

```
# ==============================================
# 9.4 Top Consensus Features
# ==============================================
ax4 = axes[1, 0]

top_features = top_consensus[:15]
feature_names_short = [
    f[0][:40] + '...' if len(f[0]) > 40 else f[0]
    for f in top_features
]
vote_scores = [f[1] for f in top_features]

bars = ax4.barh(range(len(feature_names_short)), vote_scores)

ax4.set_yticks(range(len(feature_names_short)))
ax4.set_yticklabels(feature_names_short)
ax4.set_xlabel('Number of Methods')
ax4.set_title('Top 15 Consensus Features')
ax4.set_xlim([0, max(vote_scores) + 1])
```

```
(0.0, 6.0)
```

```
# ==============================================
# 9.5 Feature Reduction Pie
# ==============================================
ax5 = axes[1, 1]

original = X_train_bal.shape[1]
consensus = len(valid_consensus_features)

labels = ['Original Features', f'Consensus\n({consensus})']
sizes = [original, consensus]

wedges, texts, autotexts = ax5.pie(
    sizes,
    labels=labels,
    autopct='%1.1f%%',
    startangle=90,
    explode=(0.05, 0.05)
)

ax5.set_title(f'Feature Reduction: {(1-consensus/original)*100:.1f}%')
```

```
Text(0.5, 1.0, 'Feature Reduction: 9.0%')
```

```
# ==============================================
# 9.6 AUC vs PR-AUC Scatter
# ==============================================
ax6 = axes[1, 2]

scatter = ax6.scatter(
    consensus_results_df['auc'],
    consensus_results_df['pr_auc'],
    s=150,
    alpha=0.7
)

ax6.set_xlabel('AUC-ROC')
ax6.set_ylabel('PR-AUC')
ax6.set_title('AUC vs PR-AUC Trade-off')
ax6.grid(alpha=0.3)

for _, row in consensus_results_df.iterrows():
    ax6.text(
        row['auc'],
        row['pr_auc'],
        row['model'],
        fontsize=8,
        alpha=0.8
    )

plt.tight_layout()
plt.show()
```

```
<Figure size 640x480 with 0 Axes>
```